The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

17

Administration: The Dashboard, and What to Watch

Part V — Running and Extending the Shop

Taught in	Week 13
Laboratory	A refund through the order state machine
Assessment	**Assignment 2 set**

This is a class extract. It is *Chapter 17* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 17

Administration: The Dashboard, and What to Watch

Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** what each dashboard widget counts, and **state** the filter behind it.
- **Explain** why the dashboard's numbers disagree, and **reconcile** them against the underlying table.
- **Distinguish** a figure that describes the shop from a figure that would change a decision.
- **Derive** three measurements this course has produced that the dashboard does not show, and **write** the query for each.
- **Design** a panel for a stated shop, and **justify** every number on it.

Before You Start

Chapter 10: order statuses, and status 0. Chapter 6: reading the query a screen actually runs. Chapter 11 and Chapter 13: the costs that make some numbers worth watching and others not.

Key Terms in This Chapter

dashboard • widget • report • filter • vanity metric • actionable metric • reconciliation • abandonment rate • conversion • leading and lagging indicator • denominator

17.1 Eight Widgets, and What Each One Counts

The administrator's first screen carries eight panels. They are extensions like any other, each switched on at **Extensions › Extensions › Dashboard**, each with a width and a sort order.

Widget	Shows	Worth knowing
`order`	Orders today, yesterday, in total	Counts every order above status 0
`sale`	Money, over the same periods	Same filter, different query
`customer`	Registered customers	Not buyers — registrations
`online`	Visitors active now	**Empty unless you switch on tracking**
`activity`	What customers have been doing	**Also empty by default**
`chart`	Orders over time	**Counts only completed statuses**
`map`	Where orders came from	By billing country
`recent`	The last few orders	The most useful of the eight

Two of the eight are permanently empty, and that is a setting

`config_customer_online` and `config_customer_activity` are both **0** in a fresh install, so the `online` and `activity` panels have nothing to show and never will until somebody changes them.

Before switching them on, recall Chapter 9. Both work by writing a row for each visitor's activity — so turning them on adds a database write to page views that previously had none, on a shop whose session table already accumulated 645 rows without a single customer. The panels are not free; they are paid for in writes on your busiest pages.

Decide what you would do differently if you knew. If the answer is "nothing", leave them off. That question is the whole of this chapter.

17.2 The Numbers Disagree, and Nothing Says So

The four headline figures are produced by different queries. This is not a fault — each filter is right for its own purpose — but the screen presents them side by side with no indication that they are answering different questions.

Worked Example:

A known set of orders. Produced by `code/m17_dashboard.py`: five orders paid for through the checkout, two of which were then moved to **Complete**, plus one abandoned at the payment button.

What is actually in `oc_order`:

Status	Orders	Value	
0	1	854.40	*not a status at all*
1	3	2,563.20	Pending
5	2	1,708.80	Complete

What each part of the dashboard reports of exactly that data:

	Orders	Value	Filter
Orders widget	5	4,272.00	status > 0
Sales widget	5	4,272.00	status > 0
The chart	**2**	**1,708.80**	status in $(5, 3)$
The table itself	6	5,126.40	everything

Three different answers to "how many orders", all correct. Six in the table, five on the widgets, two on the chart — and a shopkeeper looking at the screen sees "5" above a chart whose bars add up to 2, with nothing explaining the difference.

Both gaps are information, once you know they are there.

Six minus five is the abandonment. One order in six reached the payment button and stopped. That is the single most useful number a small shop can compute, and it exists only as the difference between two figures the dashboard shows separately.

Five minus two is the work not yet done. Three orders are sitting at Pending: paid for, not shipped. On a Monday morning that is the day's job. On a Friday it is a problem.

And the chart's filter is a *setting*, not a constant: `config_complete_status` is `["5","3"]` here, so Shipped counts as complete. Change it and every historical bar changes with it.

How to read any dashboard you did not build

1. **For each number, ask what it counts and what it excludes.** The answer is in a query, and you now know how to find it.
2. **Reconcile against the table.** If the parts do not add up to the whole, the difference has a name — and the name is usually the interesting number.

3. **Find which filters are settings.** Those can change under you without any code changing.
4. **Check the denominator.** "Conversion" means nothing until you know whether the bottom of the fraction is visitors, sessions, baskets or people.

17.3 What the Dashboard Does Not Show

Every measurement in this course that would change a decision is absent from the administrator's first screen. That is not an accident: the dashboard shows what is cheap to count, and those are not the same things.

</> Three panels this course has already earned

1. **Abandonment rate.** From Chapter 10: orders at status 0 against orders above it, by day. `SELECT DATE(date_added) d, SUM(order_status_id=0) abandoned, SUM(order_status_id>0) placed FROM oc_order GROUP BY d ORDER BY d DESC;` A shop with a rising ratio has a checkout problem, a payment problem or a price problem, and knows to look before customers tell it.
2. **Products that cannot be delivered profitably.** From Chapter 11: enabled products with a weight of zero, which under weight-based shipping are quoted free delivery. `SELECT COUNT(*) FROM oc_product WHERE status=1 AND weight=0;` A number that should be zero and silently is not.
3. **Orders standing still.** From this chapter: how long orders have been at Pending. `SELECT order_status_id, COUNT(*) n, MIN(date_added) oldest FROM oc_order WHERE order_status_id>0 GROUP BY order_status_id;` The widgets say how many orders arrived. Nothing says how many are waiting, or for how long — which is the only one of the three a shopkeeper can act on this morning.

All three are one query each, and all three are Chapter 19's subject: a dashboard widget is the smallest useful module you can write.



Figure 17.1: A test for every number you are thinking of putting on a screen. All eight shipped widgets sit at the left-hand end.

Common Pitfalls

- **Comparing two dashboard numbers that have different filters.** Five orders and a chart showing two is not a bug report.
- **Reporting revenue from a table count** without excluding status 0.
- **Switching on the online and activity panels** without noticing the write they add to every page view.
- **Treating `config_complete_status` as a constant.** It is a setting, and changing it rewrites history.
- **Adding a number nobody would act on.** Every panel costs a query on the busiest screen in the admin panel.
- **Quoting a conversion rate without naming its denominator.**
- **Watching totals rather than ages.** "How many orders" is a report; "how long has this one been waiting" is a job.

Try It Yourself: Make the dashboard disagree with itself, then fix the gap

```
# --- 1. Build a known set of orders ---------------------------------
cd code
python m17_dashboard.py

# --- 2. Now look at the admin dashboard -----------------------------
# Admin: Dashboard. Write down the four headline numbers and what the
# chart shows. Compare with what step 1 printed.

# --- 3. Find the filter behind each number --------------------------
docker compose exec shop sh -c \
  'grep -n "order_status_id" admin/model/sale/order.php | head -3'
# Then do the same for the chart's model. Three different filters.

# --- 4. Account for both gaps ---------------------------------------
# 6 in the table, 5 on the widgets, 2 on the chart. Name each gap.

# --- 5. Change a setting and watch history move ---------------------
# Admin: System > Settings > Edit > Option. Add "Processing" to the
# complete order statuses. Reload the dashboard. Which numbers moved?

# --- 6. Write the three queries the dashboard does not have ---------
# Abandonment by day; enabled products with zero weight; orders by
# status with the age of the oldest. Run each one.

# --- 7. Decide what you would put on the screen ---------------------
# Six panels, no more. For each: the number, the query, and one
# sentence saying what you would DO if it moved.

# --- 8. Clean up ----------------------------------------------------
python m17_dashboard.py --clean
```

What to notice. Step 7 is harder than it looks. Most proposed panels fail the "what would you do" test, and the ones that survive are rarely the ones that were easiest to count.

Checkpoint — Test Yourself

1. The table holds 6 orders worth 5,126.40; the widgets report 5 orders worth 4,272.00; the chart shows 2. Explain each gap, say which of the three figures you would put in front of the shop's owner, and what you would label it.

2. A shop owner says "our conversion rate is 3%". List the questions you would ask before accepting that number, and explain why two shops both reporting 3% may not be comparable at all.
3. Propose a dashboard panel that would have caught Chapter 10's broken guest checkout — where every page returned 200 and no order could be placed. State the query, how often it runs, and the condition that raises the alarm.

Chapter Summary

- Eight widgets ship. Two of them — online and activity — are empty until tracking is switched on, and that tracking costs a database write on page views that previously had none.
- Worked: the same six orders were reported as 6, 5 and 2 by the table, the widgets and the chart. All three are correct; none is labelled with its filter.
- The gaps are the information. Table minus widgets is the abandonment rate; widgets minus chart is the work not yet done.
- The chart's filter is `config_complete_status`, a setting — so changing it rewrites every historical bar.
- To read any dashboard: ask what each number excludes, reconcile against the table, find which filters are settings, and check the denominator.
- A number earns its place on a screen only if somebody would do something differently on seeing it move. Most shipped widgets describe; few change a decision.
- Three panels this course has already earned, one query each: abandonment by day, enabled products with zero weight, and orders by status with the age of the oldest.
- Totals are a report; ages are a job. Prefer the number that names today's work.

Review Questions

1. (MCQ) The dashboard chart counts orders whose status is: (a) greater than zero; (b) in `config_complete_status`; (c) any status; (d) Pending only.

2. (MCQ) The difference between the order count in the table and the order count on the widgets is: (a) a bug; (b) refunded orders; (c) orders abandoned at the payment button; (d) orders from other stores.

3. (Short) Explain why switching on the "customers online" panel is not free, referring to a finding from Chapter 9.

4. (Short) Give the test that decides whether a number belongs on a dashboard, and apply it to "total registered customers".

5. (Short) Why does changing `config_complete_status` alter charts of months that have already happened?

6. (Applied) A shop's orders widget has been flat for three weeks while its sales widget has fallen 20%. Give three distinct explanations consistent with both, say what you would query to distinguish them, and state which explanation you would investigate first and why.

7. (Applied) Design the dashboard for a shop taking 40 orders a day on cash on delivery. Give six panels, the query behind each, how often it refreshes, and for every one a sentence beginning "if this moves, we will…". Justify anything you left off that the shipped dashboard includes.